AI in the app

On-device ML, custom models, and safe cloud LLM calls

Dinu-Ștefan Rusu
FILS, Universitatea Politehnica București · Autumn 2026

TODAY

What you should leave with

On-device or cloud

The variables that decide it: latency, privacy, offline behavior, cost per call, capability ceiling.

ML Kit and LiteRT

Real Dart for text and barcodes, plus your own quantized model through tflite_flutter.

Where the key lives

Why a key inside an app binary is public, and the two architectures that fix it.

LLMs in a real UI

Streaming answers from a large language model, timeouts, token cost, and validating output you cannot trust.

PART 1 · THE DECISION

On the device, or on someone else's GPU

Five variables: latency, privacy, offline, cost, capability ceiling

DECISION

Three different things called AI in the app

Classic on-device ML

A fixed-purpose machine learning (ML) model runs locally: read text, find a barcode, locate a face. Milliseconds, offline, nothing leaves the phone.

On-device generative

A small language model on the phone's own accelerator: summarize, proofread, rewrite. Gated on hardware, never guaranteed.

A cloud large language model

A large model on someone else's machines. Highest capability, network-bound, billed per token.

Artificial intelligence (AI) in an app is not one thing. The three shapes differ in where the compute happens, what happens with no network, and what an attacker can reach.

Most shipping apps use two of the three. Picking one is an architecture decision, not a library choice, and it is expensive to reverse late.

DECISION

On-device vs cloud, variable by variable

	ON-DEVICE	CLOUD
Latency	a few milliseconds	hundreds of ms to seconds, plus the round trip
No connection	works, offline is normal	the feature does not exist
Where data goes	stays on the device	to a third party
Cost per call	zero, forever	billed per token, forever
Capability ceiling	limited by memory	the largest model in production
Updating the model	ships with an app update	instant, server-side

On-device first, cloud when it cannot cope. Privacy, offline behavior and cost all point one way. Only capability points the other.

DECISION

Will the model even fit on the phone

28 GB

7 billion parameters at float32, 4 bytes each

14 GB

the same model at float16

About 1 GB

a 2 billion parameter model at int4, phone-class

Parameter count times bytes per parameter is memory you must find before a single token appears. A 2026 flagship phone has roughly 8 to 16 GB of random-access memory (RAM), shared with the system and every other app.

On-device generative AI is a resource-allocation problem. Mobile and Embedded Computing covers the full memory arithmetic.

DECISION

The pattern that ships: hybrid

1. Input arrives. A camera frame, a document, a question.
2. The on-device model runs first. Free, offline, private, milliseconds.
3. Is it good enough? Measured against a threshold you set.
4. Escalate to the cloud. Only for what the small model cannot do.

Scan locally, look up remotely.
ML Kit reads the barcode on the device; only the resulting digits go to your own application programming interface (API).

Draft locally, refine remotely.
The on-device model writes a first version instantly; the cloud model runs only when the user asks for better.

DECISION

Know your escalation rate

Detect locally, understand remotely

A face or object detector finds the region; you upload a small crop instead of a full-resolution frame.

Cache, then do not call at all

The same barcode, photo or question asked twice becomes zero latency and zero cost the second time.

The share of requests that reach the cloud is one number.

It is your bill, your slowest response time, and your privacy exposure, all at once.

A hybrid architecture is a promise you can break silently. A system whose escalation rate nobody can quote is a cloud system with a cache in front of it.

PART 2 · ON THE DEVICE

ML Kit, LiteRT, and making a model fit

The problems already solved, and then your own

Google ML Kit: the everyday building blocks

Text recognition

google_mlkit_text_recognition reads Latin, Chinese, Japanese, Korean and Devanagari script, as blocks of lines of elements.

Barcode scanning

google_mlkit_barcode_scanning reads every common 1D and 2D symbology, plus the parsed payload for a URL or a Wi-Fi network.

Face detection

google_mlkit_face_detection returns bounding boxes, landmarks and a smile probability. Detection, never recognition.

Machine learning (ML) here means a model Google already trained and shipped for you. Nothing on this slide trains on your data or learns from your users.

More ML Kit, and how you add it

Language ID and translation

google_mlkit_language_id and google_mlkit_translation ship per-language-pair models the device downloads once and keeps.

Labeling and object detection

google_mlkit_image_labeling gives coarse categories; google_mlkit_object_detection tracks a region across frames.

How you add it

One plugin per feature, Android and iOS only. The google_ml_kit umbrella pulls every model into your binary at once.

These are community-maintained Dart wrappers. Each call crosses a platform channel into the native ML Kit SDK. Fine for a photo; it is why the next slide is about back-pressure, not raw speed.

Reading text from an image, end to end

```dart
// pubspec: google_mlkit_text_recognition
final _reader = TextRecognizer(
    script: TextRecognitionScript.latin);

Future<String> readText(String path)
    async {
  final input =
    InputImage.fromFilePath(path);
  final result =
     await _reader.processImage(input);
  return result.text;
}

void dispose() => _reader.close();
```

Runs off the Dart thread.
The UI keeps rendering while a frame is read.

The result is a tree.
Blocks of lines of elements, each with a bounding box.

close() is not optional. The detector owns a native model; skipping it is a leak the Dart garbage collector cannot see.

Barcodes from a live camera stream

```
bool _busy = false;
Future<void> onFrame(CameraImage f)
    async {
  if (_busy) return; else _busy = true;
  final codes = await
    _scanner.processImage(_toInput(f));
  if (codes.isNotEmpty)
    onCode(codes.first);
  _busy = false;
}
```

A camera delivers about 30 frames a second.
Inference will not always finish inside 33 ms.
Dropping frames is correct: the next arrives in a moment.

Real-time on-device ML is a back-pressure problem. The model is fast enough. The frame pipeline breaks first.

On-device generative AI is a capability

The step past classification: a small language model on the phone's own neural processing unit (NPU) can summarize a thread, proofread a message, or rewrite a paragraph.

Google exposes this through the ML Kit generative AI APIs, backed by Gemini Nano; Apple exposes an equivalent on supported hardware. Both are the same shape from your side.

Ask the SDK. Three answers.

1. **Available.** Run it locally.
2. **Downloadable.** Ask, show progress, or defer.
3. **Unsupported.** Fall back to the cloud.

Feature-detect, never device-detect. A hard-coded list of phone models goes stale fast. Availability is a runtime question, and the fallback is part of the feature.

Your own model: LiteRT, formerly TensorFlow Lite

ML Kit covers the problems Google already solved. When the task is yours, such as grading a weld, you bring your own model.
LiteRT executes a .tflite file on the device: the graph plus the weights, with fixed input and output shapes.
tflite_flutter binds the LiteRT C API through dart:ffi: a direct native call, not a platform-channel message.

1. **Train.** TensorFlow or PyTorch, on a real GPU.
2. **Convert.** To a .tflite file.
3. **Quantize.** float32 to int8, about four times smaller.
4. **Ship and run.** As an asset or a download, then Interpreter.

None of the first three steps happen in Dart. A delegate hands layers to an accelerator; every delegate may refuse, so the CPU is always the fallback.

tflite_flutter: load, run, get off the UI isolate

```
late final Interpreter _interp;
late final IsolateInterpreter _iso;
Future<void> load() async {
  _interp = await Interpreter.fromAsset(
      'assets/leaf_int8.tflite');
  _iso = await IsolateInterpreter.create(
      address: _interp.address);
}
Future<List<double>> classify(List x) async {
  final out =
      List.filled(5, 0.0).reshape([1, 5]);
  await _iso.run([x], out);
  return out[0].cast<double>();
}
```

run is synchronous and CPU-bound.
On the UI isolate it drops frames, so hand the native address to an isolate.

Shapes are fixed at conversion time; guessing them produces a crash or silent nonsense.

close() both. Native memory again, invisible to the Dart garbage collector.

Quantization: shrinking the model to fit

	FLOAT32	FLOAT16	INT8
Size	baseline	half	a quarter
Accuracy loss	none	usually negligible	small, so measure it
CPU speed	baseline	about baseline	2 to 4 times faster

Bundled as an asset, the model works on first launch. Downloaded on first run, the install is smaller but you now own a cache, a version check and a retry.

Accelerator paths are int8-first. Quantization is often the difference between running on the NPU and falling back to the CPU.

PART 3 · CLOUD LLMS

Calling a model you do not host

Where the key lives decides the architecture

The prototyping trap: a key inside the app

```
// google_generative_ai is deprecated.
const apiKey =
    'AIzaSyD-EXAMPLE_KEY_xxxxxxxxxx';

final model = GenerativeModel(
  model: 'gemini-2.5-flash',
  apiKey: apiKey,  // compiled into the app
);

final res = await model.generateContent(
    [Content.text(prompt)]);
```

The package.
google_generative_ai is deprecated; firebase_ai replaces it, and fixes the other two problems too.

The key.
Anything compiled into your app ships to every person who installs it. A .env file or base64 changes nothing.

The architecture.
The client talks straight to a metered API, so nothing but the client limits who spends against your account.

A key in a binary is a published key

```
# Anyone can run this, on their own phone.
$ adb pull /data/app/~~a1b2/base.apk .
$ unzip -o base.apk -d out/
$ strings out/lib/arm64-v8a/libapp.so \
    | grep -E 'AIza[0-9A-Za-z_-]{35}'
AIzaSyD-EXAMPLE_KEY_xxxxxxxxxx

# Elapsed: about a minute.
# iOS: the same exercise on the .ipa file.
```

Release Dart is compiled ahead of time into native code, but string constants survive: the program has to read them at runtime.

Obfuscation renames symbols. It does not remove the bytes your HTTP client puts on the wire.

The bill is yours. A leaked key is billed to the account that issued it until you notice.

There is no secret you can ship inside a client. The same rule as the authentication lecture, this time with a billing consequence.

Shape A: your backend proxies the call

1. Flutter app. Sends the user's ID token, never a key.
2. Your backend. Verifies the token, holds the provider key.
3. Model provider. Billed to you, called only by you.

Works with any provider, with your own rate limits, your own logging and your own prompt control, at the cost of a service you now have to operate and keep online.

The credential lives somewhere the user cannot reach. That single property is what separates this from the key-in-the-app version.

Shape B: Firebase AI Logic and App Check

1. Flutter app. Uses firebase_ai, with no key to pass.
2. Firebase AI Logic. App Check attests the caller is a genuine build.
3. Gemini. Billed to your own Firebase project.

Working the same afternoon, with attestation built in, but Google models only, and only once you press Enforce in the App Check console.

Both shapes hide the credential from the user. Pick A for provider freedom, B for the fastest path to something that cannot be extracted from the app.

firebase_ai: the call with no key in it

```
// Firebase.initializeApp already ran
    (Lecture 8)
await FirebaseAppCheck.instance.activate(
  androidProvider:
    AndroidProvider.playIntegrity,
  appleProvider: AppleProvider.appAttest,
);
final model = FirebaseAI.googleAI()
    .generativeModel(
  model: 'gemini-2.5-flash',
  generationConfig: GenerationConfig(
      temperature: 0.2, maxOutputTokens:
    512),
);
```

No apiKey parameter exists.
The credential is the project configuration; App Check gates the request.

activate() only measures.
Enforcement is a separate switch you flip in the console, per product.

Stream the answer, do not freeze the screen

```
Stream<String> ask(String prompt) async*
    {
  final buf = StringBuffer();
  final s = _model.generateContentStream(
      [Content.text(prompt)]);
  await for (final chunk in s) {
    buf.write(chunk.text ?? '');
    yield buf.toString();   // UI
     rebuilds per chunk
  }
}
```

Time to first token is what users feel. Streaming turns a six-second wait into a 600 ms one.

Feed this into a StreamBuilder. Show three states: sent, thinking, writing, since a spinner that never changes looks like a hang.

A frozen screen is a bug, not a wait. Cancel the stream on dispose or when a new question starts.

PART 4 · ENGINEERING REALITY

Cost, failure, and answers that are wrong

Three areas to get right before you ship

ENGINEERING REALITY

Tokens: the unit you are billed in

About 4

characters per token, for English prose

In and out

both directions are billed, every turn

n^2

cost of a naive chat: n turns cost roughly n squared

Cap the output with maxOutputTokens, and cap the input by summarizing history instead of resending all of it. Route by difficulty: a small model answers most questions, only its failures reach the expensive one.

Per-user quotas live on your backend. An app cannot enforce a quota against a user who controls the app: the same rule as the authentication lecture.

ENGINEERING REALITY

The failures you will actually see

CODE	WHAT IT MEANS	WHAT TO DO
429	Rate limited, or over quota	Back off with jitter, capped
503 / 500	The provider is failing	Retry, but the retry is billed too
400	Your request is wrong	Never retry; fix the request
Timeout	No answer in budget	Cancel, say so, offer a retry
Safety block	The model declined	A normal state, not an exception

Set a budget for the whole feature, not one attempt: three tries of ten seconds each is a spinner no user waits out.

Retries multiply load and cost, so cap them. Reuse the same backoff-with-jitter machinery from the networking lecture.

ENGINEERING REALITY

Models produce plausible wrong answers

Not a bug you can prompt away.
A model returns a likely continuation, not a true one.

Constrain the output space.
A fixed set of values cannot be hallucinated into something new.
Validate anyway: parse it, range-check it, look every id up in your own data.

```
responseMimeType:
    'application/json',
responseSchema:
    Schema.object(properties: {
  'category': Schema.enumString(
      enumValues: ['food',
    'travel']),
}),
```

⚠ **Never wire model output straight to consequence**

Money, a database write, a message to another person: a rule or a human belongs in between.

RECAP

Three things to remember

1. **On-device or cloud is an architecture decision.** Latency, privacy, offline behavior, cost and capability ceiling, not a library choice.
2. **No app ever holds a provider key.** Your backend holds it, or Firebase AI Logic with App Check does.
3. **Stream the answer, cap the tokens, and validate everything the model says.** Treat model output like input from the internet.

RECAP

Further reading

FIREBASE AI LOGIC

firebase.google.com/docs/ai-logic

GOOGLE ML KIT

developers.google.com/ml-kit

LITERT

ai.google.dev/edge/litert

TFLITE_FLUTTER ON PUB.DEV

pub.dev/packages/tflite_flutter

Questions?

dinu_stefan.rusu@upb.ro · Microsoft Teams: course channel